SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA0307	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	Charandeep.N.C	**Reg. No.:**	192472196
Section / Batch:	CSE-AI	**Date:**	8-8-2026

Code of Conduct

I ***Charandeep.N.C(192472196)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: **Charandeep**

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System

Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

Annexure A – Case Study

CASE STUDY 1: Smart Mobile Keyboard Prediction System

A smartphone company is developing an AI-powered keyboard application that predicts the next word while users type emails, messages, and social media posts. The language model is trained on the corpus:

"Data science is powerful, data science drives innovation, data science is evolving."

The development team employs Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis to improve prediction accuracy and handle unseen word sequences.

The following statistics are obtained from the corpus:

- C(data) = 3
- C(data science) = 3
- C(science is) = 2
- C(science drives) = 1
- C(science drives innovation) = 1

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word "science" are:

- P(is) = 0.66
- P(drives) = 0.33

Introduction:
The Smart Mobile Keyboard Prediction System uses Natural Language Processing (NLP) techniques such as N-gram language models, Maximum Likelihood Estimation (MLE), Backoff Models, Deleted Interpolation, and Entropy Analysis to predict the next word accurately. These techniques improve prediction accuracy, handle unseen word sequences, and enhance the typing experience by providing intelligent real-time suggestions.

Questions

1. **Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability P(science | data). Interpret how this probability influences next-word prediction in a mobile keyboard application.**

Answer:

Formula:

- P(science | data) = C(data science) / C(data) **Calculation:**
- P(science | data) = 3 / 3 = 1.0 **Result: P(science | data) = 1.0 (100%) Interpretation:**
- In this training corpus every occurrence of “data” is followed by “science”, so the MLE assigns full probability (1.0) to this transition.
- For the mobile keyboard, this means “science” would be suggested with very high confidence immediately after the user types “data”.
- A probability of exactly 1 also signals data sparsity (a very small training corpus) rather than absolute certainty in real usage — this is why smoothing/backoff/interpolation techniques are still needed alongside MLE.

2. **A user types the unseen sequence "data science improves". Apply the Backoff Model and estimate the probability using lower-order n-grams. Explain why backoff is essential for realtime predictive text systems.**

Answer:

- Given: the trigram “science improves” never occurs in the corpus, so its MLE count = 0 (an unseen sequence).

Backoff Procedure:

- Step 1: Try the highest-order estimate P(improves | data, science) — count = 0, cannot be used.
- Step 2: Back off to the bigram P(improves | science) — also 0 in this corpus.
- Step 3: Back off further to the unigram estimate P(improves), scaled by a back-off weight α, instead of assigning probability 0.
- Formula: P_backoff(wi | wi-2, wi-1) = P(wi | wi-2, wi-1) if count > 0; otherwise α(wi-2, wi-1) × P_backoff(wi | wi-1)

Why backoff is essential:

- Users constantly type novel word combinations that never appeared in the training data; without backoff such sequences would receive probability 0 and prediction would fail completely.
- Backoff guarantees every sequence keeps a small, non-zero probability by falling back to more general, reliably-estimated lower-order n-grams, keeping real-time prediction robust to sparse data.

3. Using Deleted Interpolation, calculate the probability of the sequence "data science is". Discuss how interpolation balances reliability and coverage in sparse datasets.

Answer:

Additional counts needed (from the corpus, N = 12 total word tokens):

- C(science) = 3, C(data science is) = 2 (from "data science is powerful" and "data science is evolving"), C(is) = 2

Component probabilities:

- Trigram: P(is | data, science) = C(data science is)/C(data science) = 2/3 = 0.667
- Bigram: P(is | science) = C(science is)/C(science) = 2/3 = 0.667
- Unigram: P(is) = C(is)/N = 2/12 = 0.167
- Deleted Interpolation formula: P_interp(w3|w1,w2) = λ1×P_tri + λ2×P_bi + λ3×P_uni
- = (0.5×0.667) + (0.3×0.667) + (0.2×0.167) = 0.334 + 0.200 + 0.033 = 0.567
- Sequence probability: P(data science is) = P(data) × P(science|data) × P_interp(is|data,science)
- = (3/12) × 1.0 × 0.567 = 0.25 × 1.0 × 0.567 ≈ 0.142

Result: P("data science is") ≈ 0.142 Discussion:

- Deleted interpolation blends the specific but sparse trigram estimate with the more general and stable bigram and unigram estimates using fixed weights.
- This balances reliability (trigram captures the most context when enough data exists) with coverage (unigram/bigram ensure every sequence still gets a reasonable non-zero probability even when trigram data is sparse).

4. Calculate the Entropy of the distribution P(is)=0.66 and P(drives)=0.33. Interpret the result and evaluate how entropy helps measure uncertainty and prediction confidence in intelligent keyboard systems.

Answer:

- Formula: H = −Σ P(x) log2 P(x)
- H = −[0.66 × log2(0.66) + 0.33 × log2(0.33)]
- log2(0.66) ≈ −0.599, log2(0.33) ≈ −1.599
- H = −[0.66×(−0.599) + 0.33×(−1.599)] = 0.395 + 0.528 = 0.923 bits

Result: H ≈ 0.92 bits Interpretation:

- The maximum possible entropy for 2 outcomes is 1 bit (perfect 50/50 uncertainty). The computed value (≈0.92 bits) is close to this maximum, showing the model is only mildly more confident about "is" than "drives".

- High entropy = low prediction confidence; the keyboard should show multiple candidate words rather than auto-committing one. Lower entropy would indicate a single dominant, highly confident prediction.

CASE STUDY 2: AI-Powered Customer Support Chatbot

A multinational airline deploys a customer support chatbot that processes user messages and automatically identifies the grammatical role of words before generating responses.

Consider the following user inputs:

1. "Book a flight ticket now."
2. "This book is interesting."

The chatbot uses the Penn Treebank POS Tagset and a Hidden Markov Model (HMM) for probabilistic tagging.

Given:

- P(book | VB) = 0.6
- P(book | NN) = 0.4
- P(Start → VB) = 0.5
- P(Start → NN) = 0.5

The system must determine whether the word "book" functions as a Verb (VB) or Noun (NN) depending on context.

Questions

1. **Assign appropriate Penn Treebank POS tags to every word in both sentences. Justify the tag assigned to the word "book" using contextual clues.**

Answer:

Sentence 1: "Book a flight ticket now."

- Book/VB a/DT flight/NN ticket/NN now/RB ./.

Sentence 2: "This book is interesting."

- This/DT book/NN is/VBZ interesting/JJ ./.

Justification for "book":

- Sentence 1 is an imperative command with no explicit subject; "Book" is the first word and functions as the action ("to reserve") → tagged VB (base-form verb).
- Sentence 2 has "book" preceded by the determiner "This" and followed by the linking verb "is", so it functions as the subject of the sentence → tagged NN (noun).

- The surrounding context (sentence position, determiner, following verb) resolves the lexical ambiguity of “book”.

2. **Using the HMM probabilities, compute the likelihood of "book" being tagged as a Verb (VB) in the sentence "Book a flight ticket now." Explain why the probabilistic model favors this tag.**

Answer:

- Given: P(book|VB)=0.6, P(book|NN)=0.4, P(Start→VB)=0.5, P(Start→NN)=0.5 **HMM Likelihood (emission × transition):**
- Score(VB) = P(Start→VB) × P(book|VB) = 0.5 × 0.6 = 0.30
- Score(NN) = P(Start→NN) × P(book|NN) = 0.5 × 0.4 = 0.20
- Normalized: P(VB|book) = 0.30/0.50 = 0.6 (60%), P(NN|book) = 0.20/0.50 = 0.4 (40%)

Result: Since 0.30 > 0.20, the HMM tags “book” as VB. Explanation: • The HMM chooses the tag with maximum joint probability of transition and emission. Both the higher emission probability P(book|VB)=0.6 and the sentence-initial (Start) position favour VB, correctly matching the imperative structure of “Book a flight ticket now.”

3. **Compare Rule-Based Tagging and Stochastic (HMM) Tagging in resolving lexical ambiguity. Recommend which approach is more suitable for large-scale chatbot deployment and justify your answer.**

Answer:

Rule-Based Tagging:

- Uses hand-written linguistic rules and context patterns.
- Transparent and easy to interpret/debug.
- Poor scalability — struggles with unseen patterns and needs constant manual rule updates. **Stochastic (HMM) Tagging:**
- Learns emission and transition probabilities automatically from large training corpora.
- Resolves ambiguity statistically and adapts as more data is added.
- Scales well and generalizes to unseen sentences.

Recommendation:

- Stochastic (HMM) tagging is more suitable for large-scale chatbot deployment, since it scales automatically across large and evolving conversational data, requires no manual rule maintenance for every new ambiguous case, and provides probabilistic confidence scores useful for downstream intent detection.

4. **Evaluate the role of standardized POS tagsets and word classes in improving intent detection, response generation, and overall chatbot performance.**

Answer:

- Standardized tagsets (e.g., Penn Treebank) give a consistent, shared vocabulary of grammatical categories so every module of the chatbot pipeline interprets words the same way.
- Correct word-class identification helps determine user intent — e.g., "book" as VB signals a booking action, while "book" as NN signals an object being discussed.
- Improves accuracy of downstream tasks such as intent classification, entity/slot extraction, and response-template selection.
- Reduces ambiguity-driven errors, leading to more relevant, context-appropriate automated responses and better overall chatbot performance and customer satisfaction.

CASE STUDY 3: News Analytics and POS Tag Correction System

A digital news analytics company processes millions of news articles daily. The platform uses a Transformation-Based Tagger (Brill Tagger) to improve tagging accuracy after an initial POS tagging phase.

The sentence processed by the system is: "Economic

growth increases employment."

Initial POS Tags:

- economic/JJ
- growth/NN
- increases/NNS
- employment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and uncertainty evaluation.

Questions

1. **Apply the transformation rule and update the POS tag sequence. Explain why the correction is linguistically appropriate.**

Answer:

- Initial tags: economic/JJ growth/NN increases/NNS employment/NN
- Rule: Change NNS → VBZ if the preceding word is tagged NN.
- The word preceding "increases" is "growth/NN" → condition is satisfied. **Updated tags: economic/JJ growth/NN increases/VBZ employment/NN Why the correction is linguistically appropriate:**
- In "Economic growth increases employment," "increases" is the main verb of the sentence (subject "growth" + verb "increases" + object "employment"), not a plural noun. Changing NNS to VBZ aligns the tag with its true grammatical function (3rd person singular, present-tense verb).

2. **Analyze the initial tagging errors and justify the corrected tag assignments using grammatical structure and sentence semantics.**

Answer:

Initial tagging error:

- "increases" was mistagged as NNS (plural noun), most likely because the initial tagger matched the surface form to its more frequent nominal usage without using sentence context.
- This produces an ungrammatical tag sequence JJ+NN+NNS+NN — two nouns ("growth", "increases") appear consecutively with no finite verb, which is invalid for a complete English sentence.

Justification of corrected tags:

- The corrected sequence JJ+NN+VBZ+NN forms a valid Subject (growth) – Verb (increases) – Object (employment) structure, matching standard SVO sentence semantics and confirming the transformation rule's correctness.

3. **Assuming the corpus contains the words:**

    - **economic (120 occurrences)**
    - **growth (450 occurrences)**
    - **increases (210 occurrences)**
    - **employment (380 occurrences)**

    Compute the word frequency distribution and explain how frequency information supports probabilistic POS tagging.

Answer:

- Total occurrences = 120 + 450 + 210 + 380 = 1160

Word Frequency Distribution (count → relative frequency):

- economic: 120 → 120/1160 = 0.103 (10.3%)
- growth: 450 → 450/1160 = 0.388 (38.8%)
- increases: 210 → 210/1160 = 0.181 (18.1%)
- employment: 380 → 380/1160 = 0.328 (32.8%) **Explanation:** • These relative frequencies serve as unigram/prior probability estimates P(word). In probabilistic POS tagging (e.g., HMM emission probabilities P(word|tag)), such corpus counts form the basis of Maximum Likelihood Estimation.
- Words with higher frequency (growth, employment) yield more statistically reliable probability estimates, while rarer words (economic) carry greater estimation uncertainty and may require smoothing.

4. **Evaluate how Transformation-Based Tagging improves tagging accuracy compared with the initial tagging stage. Discuss how entropy can be used to measure uncertainty in tag assignments before and after rule application.**

Answer:

Improvement over initial tagging:

- The initial (lexical/statistical) tagger assigns tags mainly from word-level likelihood, so it can mistag context-dependent words (e.g., “increases” as NNS).
- Transformation-Based Tagging (Brill Tagger) applies an ordered set of learned contextual rules after the initial pass, correcting systematic errors such as NN-context verbs mistagged as plural nouns — raising overall tagging accuracy without retraining the whole statistical model.

Entropy-based uncertainty measurement:

- Before correction: the initial tagger is uncertain among NN/NNS/VBZ candidates for “increases”, so probability mass is spread across multiple tags → higher entropy (more uncertainty).
- After correction: the transformation rule deterministically fixes the tag to VBZ in this context, collapsing the probability distribution onto one tag → entropy drops close to 0, reflecting higher tagging confidence.
- Conclusion: the reduced entropy after rule application is a quantitative indicator that Transformation-Based Tagging increases both certainty and accuracy compared with the initial tagging stage.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, wellstructured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________